Douglas Fairbanks and Mary Pickford became the first two celebrities to make imprints of their hands and feet in cement at Grauman's Chinese Theatre in Hollywood, California. https://en.wikipedia.org/wiki/Grauman%27s_Chinese_Theatre

2014:

A bomb blast in Ürümqi, China, killed three people and injured 79 others. https://en.wikipedia.org/wiki/April_2014_%C3%9Cr%C3%BCmqi_attack

---

Wiktionary's word of the day:

vindication: 1. (countable) An act of asserting or maintaining; an assertion. 2. (countable) An argument, fact, piece of evidence, etc., which vindicates (“clears someone of an accusation or suspicion; justifies a belief or claim by providing evidence or proof”). 3. (uncountable) The action of vindicating; also, the state of being vindicated. 4. (countable, Ancient Rome, historical, law) A legal claim for a declaration that one is the owner of a thing or the holder of a right; an action in rem. 5. (uncountable, obsolete) 6. The action of avenging or taking revenge. 7. (rare) The action of setting free; deliverance. 8. (rare) Punishment, retribution. 9. About Word of the Day 10. Nominate a word 11. Leave feedback https://en.wiktionary.org/wiki/vindication

---

Wikiquote quote of the day:

We feel our shell keeps us safe, but it crushes us and others, and keeps out light and sun. -- Taisen Deshimaru https://en.wikiquote.org/wiki/Taisen_Deshimaru

FRED TALKS

30th April 2026

CONTENTS

A statistical approach to model evaluations

ANTHROPIC.COM · 14 SEP 2025 · SOURCE

Suppose an AI model outperforms another model on a benchmark of interest—testing its general knowledge, for example, or its ability to solve computer-coding questions. Is the difference in capabilities real, or could one model simply have gotten lucky in the choice of questions on the benchmark?

With the amount of public interest in AI model evaluations—informally called “evals”—this question remains surprisingly understudied among the AI research community. This month, we published a new research paper that attempts to answer the question rigorously. Drawing on statistical theory and the experiment design literature, the paper makes a number of recommendations to the AI research community for reporting eval results in a scientifically informative way. In this post, we briefly go over the reporting recommendations, and the logic behind them.

Recommendation #1: Use the Central Limit Theorem

Evals often consist of hundreds or thousands of unrelated questions. MMLU, for instance, contains questions as diverse as:

- Who discovered the first virus?
- What is the inverse of $f(x)=4-5x$?
- Who said that “Jurisprudence is the eye of law”?

To compute an overall eval score, each question is separately scored, and then the overall score is (usually) a simple average of these question scores. Typically, researchers focus their attention on this observed average. But in our paper, we argue that the real object of interest should not be the *observed* average, but rather the *theoretical* average across all possible questions. So if we imagine that eval questions were drawn from an unseen “question universe,” we can learn about the average score in that universe—that is, we can measure the underlying *skill*, independent of the “luck of the draw”—using statistical theory.

Ultimately, there’s still plenty that needs to play out before we fully see what the future holds for enterprise software. But it will certainly be the most dynamic and exciting period we've ever seen in software history.

[Daily article] April 30: 330 West 42nd Street

ENGLISH WIKIPEDIA ARTICLE OF THE DAY · 30 APR 2026 · SOURCE

330 West 42nd Street, also known as the McGraw-Hill Building, is a 485 -foot-tall (148 m), 33-story skyscraper in the Hell's Kitchen neighborhood of Manhattan in New York City. It was designed by Raymond Hood and J. André Fouilhoux in a mixture of the International Style, Art Deco, and Streamline Moderne styles. The exterior has numerous setbacks, blue-green terracotta ceramic tile panels, and green metal- framed windows. The lobby originally had blue and green panels, and most of the upper stories have similar floor plans. The building, constructed from 1930 to 1931, was originally the headquarters of the McGraw-Hill Companies until 1972. Since then, ownership of 330 West 42nd Street has changed several times; Deco Towers has owned the building since 1994. Moed de Armas and Shannon completely renovated the building in 2021, and the upper stories were converted into apartments starting in 2023. Designated as a city landmark, the building is listed on the National Register of Historic Places. (Full article...).

Read more: https://en.wikipedia.org/wiki/330_West_42nd_Street

Today's selected anniversaries:

1006:

SN 1006, the brightest supernova in recorded history, first appeared in the constellation Lupus. https://en.wikipedia.org/wiki/SN_1006

1789:

George Washington took the oath of office as the first president of the United States at Federal Hall in New York City. https://en.wikipedia.org/wiki/Presidency_of_George_Washington

1927:

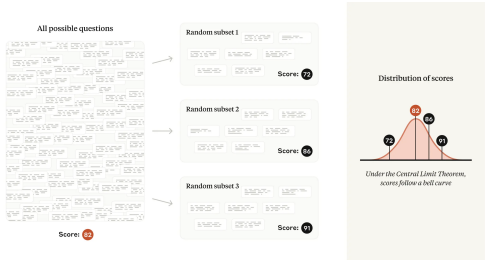
Now, the big open question is who wins in this technology wave. If we have learned anything from Clayton Christensen's Innovator's Dilemma framework, we know that incumbents tend to lose in markets when there's a new tech or business model innovation that is unattractive to them because it represents less revenue or profitability. Conversely, they tend to hold their own when the new business model or technology is supportive of their existing business, or perhaps even enhances it. Ultimately, in AI, we'll likely see examples of both outcomes.

Some existing software categories, like customer support or coding, are changing so rapidly that many incumbents will be unable to change their business models or technology stacks fast enough to respond to the growing demands of the new market, even if getting to the other side would be economically attractive. Similarly, as Jaya Gupta and Ashu Garg outlined in their piece on context graphs, there are many new categories of agent-native software that will need to be created for the first time because no existing vendor logically can capture the way the agent needs to do its work. This will create tremendous opportunities for new startups to build AI agents within these categories, and we're seeing new ones crop up every day.

Conversely, there are a number of existing SaaS players are in a natural position to power agents in their relevant work categories, especially those that have the natural context necessary for AI agents to be successful in automating work. Unsurprisingly, I'd expect there to be a correlation between the strength of a software company's moat and the amount of data they house, how complex the underlying workflows are, the number of connections there are with other systems, how intertwined a human-in-the-loop element is to the underlying agentic workflow, and how naturally AI agents can be plugged into these workflows (either as native users or via API in external agentic products).

Of course, this will mean that the business model of software (new upstarts and incumbents) will have to evolve over time. Today, the vast majority of SaaS products charge on a per-seat basis, which generally corresponds to most of the usage that the software sees today by its end users. But in a world where AI agents do most of the interaction and work on software, enterprise systems will have to evolve to support more of a consumption and usage-based model over time. AI agents don't cleanly fit as seats on software, because any given AI agent can do a varied amount of work within a system (e.g. you could have 1 agent doing a billion things or a billion agents doing one thing).

Inevitably, we'll see a mix of both outcomes. Users (or people) will continue to be seats within software, and AI agents that represent extended usage will be monetized through consumption. We're already seeing this play out in AI-native coding products and other platforms where there's an end-user seat component mixed with a consumption model on top. Obviously depending on the human vs. agent value proposition mix, the revenue stream will slide between these two ends of the continuum.



If we imagine that eval questions were drawn from a "question universe," then eval scores will tend to follow a normal distribution, centered around the average score of all possible questions.

This formulation buys us analytic robustness: if a new eval were to be created with questions having the same difficulty distribution as the original eval, we should generally expect our original conclusions to hold.

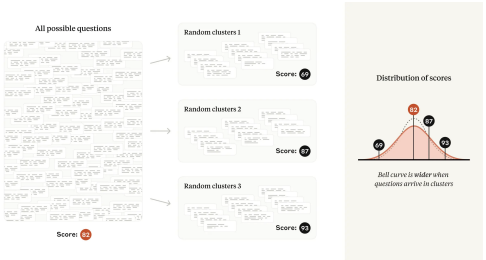
In technical terms: under the fairly mild conditions of the Central Limit Theorem, the mean values of several random samples taken from the same underlying distribution will tend to follow a normal distribution. The standard deviation (or width) of that normal distribution is commonly known as the standard error of the mean, or SEM. In our paper, we encourage researchers to report the SEM, derived from the Central Limit Theorem, alongside each calculated eval score—and we show researchers how to use the SEM to quantify the difference in theoretical means between two models. A 95% confidence interval can be calculated from the SEM by adding and subtracting 1.96 × SEM from the mean score.

Recommendation #2: Cluster standard errors

Many evals violate the above assumption of independently selected questions, and instead consist of groups of closely related questions. For example, several questions in a reading-comprehension eval may ask about the same passage of text. Popular evals that follow this pattern include DROP, QuAC, RACE, and SQuAD.

For these evals, each question's selection from the "question universe" is no longer independent. Because including several questions about the same passage of text will yield less information than selecting the same number of questions about different passages of text, a naive application of the Central Limit Theorem to the case of non-independent questions will lead us to underestimate the standard error—and potentially mislead analysts into drawing incorrect conclusions from the data.

Fortunately, the problem of clustered standard errors has been extensively studied in the social sciences. When the inclusion of questions is non-independent, we recommend clustering standard errors on the unit of randomization (for example, passage of text), and we provide applicable formulas in our paper.



If questions arrive in related clusters—a common pattern in reading-comprehension evals—eval scores will be more spread-out compared to the non-clustered case.

In practice, we have found that clustered standard errors on popular evals can be over three times as large as naive standard errors. Ignoring question clustering may lead researchers to inadvertently detect a difference in model capabilities when in fact none exists.

Recommendation #3: Reduce variance within questions

Variance is a measurement of how spread-out a random variable is. The variance of an eval score is the square of the standard error of the mean, discussed above; this quantity depends on the amount of variance in the score on each individual eval question.

A key insight of our paper is to decompose a model's score on a particular question into two terms that are added together:

- The mean score (the average score that the model would achieve if asked the same question an infinite number of times—even if the model might produce a different answer each time); and
- A random component (the difference between a realized question score and the mean score for that question).

For instance, if you ask an AI agent to generate 10X the number of outbound email and marketing messages, where will all those agents find the data to work with, and where will the leads end up going to have a human sales rep act on them? Almost certainly some form of a CRM system. Or when an AI Agent processes reviews invoices and transactions, where will they enter this data when they're done to kick off a supply chain workflow? Again, an existing or reinvented ERP system. As long as we have humans and agents interacting with one another, there will need to be deterministic software that bridges these interactions seamlessly.

And as a result of all of these new agentic users within our software systems, the size of enterprise software markets are going to increase massively.

Unconstrained software markets

Traditionally, software companies have been stuck within the constraints of existing IT budgets, which tend to tap out around 3-7% of a company's revenue. This creates an inherent upper limit for what the budget can be for software, which translates into the total addressable market of various technology categories. Now, with AI Agents, the software is actually bringing along the work with the software, which means the budget software players are going after is the total spend that goes into doing that work in the company, not just the tech to enable it. This inevitably leads to a substantial increase in TAM for most software categories whose markets were artificially held back in size previously.

For instance, a contract management vendor just a few years ago was limited by the number of attorneys within an organization that needed to manage and review contracts. However, in a world of AI agents, that cap no longer exists, where you might have agents running continuously processing and reviewing contracts and providing extend legal services for any company. The size of the legal services market in the US is somewhere around $400B, nearly 50-100X larger than the corresponding software categories in legal space. You can easily see how AI agents making the legal industry meaningfully more productive could capture multiple percentage points of that $400B in spend, thus multiplying the size of the legal software market overnight.

The same analogy holds for almost any software category that was artificially constrained by a customer's headcount, or limited software spend, previously. We're already seeing this in spaces like coding, where the AI agents in coding are generating orders of magnitude more revenue for the new coding startups than the prior IDE categories generated. And we can expect this same dynamic for most other areas of knowledge work over time, from healthcare and financial services to legal and marketing. Tools that bring along work will generate far more revenue than the prior era of tools used to.

The opportunity ahead for software

company unique (like your core product, your talent, your culture, and so on), and the context is absolutely necessary, but largely non-differentiating between firms (think all of your systems of record like payroll, IT ticket responses, ERP systems, content management platforms, and so on).

For software that handles your company's context activities, your customer will rarely notice when these things work great, but they'll certainly notice the downstream impacts when they don't. This is why you "rent" these services from software providers that do this as their day job for thousands or millions of other companies. And incidentally, much of this software helps define these underlying workflows in your organization so you don't have to -- so each company doesn't have to reinvent the wheel each on how to handle HR, IT ticketing, and so on.

Deterministic vs. non-deterministic systems

Ok, but if an enterprise is filled with 100X more AI Agents in the future, won't that eventually consume all the use-cases that software did previously?

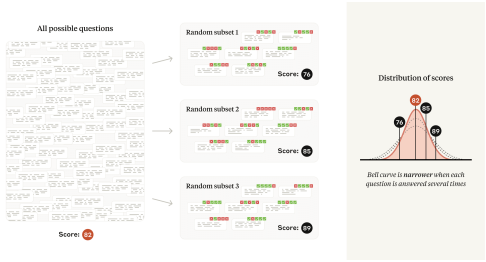
If you take the analogy of the industrial world, software is like the machinery in a manufacturing plant; and agents are like the people that operate that machinery. Just like machinery in a plant, in deterministic software, you want things to happen the same way, every time: you want data that goes into your ERP system to be calculated based on the business rules you've setup; you want security permissions you've set for accessing critical files to behave the same way every time, and not leak information between users; and you don't want an HR system probabilistically generating new reporting structures for employees on every load.

Conversely, you want *non-deterministic* systems (in this case AI agents) for the things that people have always tended to do in these systems. An agent filling out notes from a user's client meeting in a CRM system, generating a sales presentation for a new customer, answering a customer support inquiry with the best products to leverage, doing research or providing analysis on a set of enterprise documents and data for an important decision, and so on. As a result, the non-deterministic and deterministic elements of software end up working in a complementary fashion.

In contrast to some of the public discussion, I'd argue that in a world of 100X more AI agents than people in an enterprise, the value of the systems of record and tools agents will use will go up, not down. Because in this new world, software provides the guardrails on which agents can operate successfully within an enterprise, and gives them the underlying tools to use to be more productive themselves and work alongside people.

Thanks to the law of total variance, reducing the variance in the random component directly leads to a smaller standard error of the overall mean, and thus greater statistical precision. Our paper highlights two strategies for reducing variance in the random component depending on whether or not the model is asked to think step by step before answering (a prompting technique known as CoT, or chain-of-thought reasoning).

If an eval uses chain-of-thought reasoning, we recommend resampling answers from the same model several times, and using the question-level averages as the question scores fed into the Central Limit Theorem. We note that the Inspect framework correctly computes standard errors in this way via its *epochs* parameter.



If a model produces answers non-deterministically, then generating (and grading) several answers per question will result in less spread-out eval scores.

If the eval does not use chain-of-thought reasoning (i.e., its answers are not "path dependent"), we note that the random component in the score may often be eliminated altogether using next-token probabilities from the language model. For example, if the correct answer to a multiple-choice question is "B", we would simply use the probability of the model producing the token "B" as the question score. We are not aware of an open-source evals framework which implements this technique.

Recommendation #4: Analyze paired differences

Eval scores don't have any meaning on their own; they only make sense in relation to one another (one model outperforms another model, or ties another model, or outperforms a person). But could a measured difference between two models be due to the specific choice of questions in the eval, and randomness in the models' answers? We can find out with a two-sample *t*-test, using only the standard errors of the mean calculated from both eval scores.

However, a two-sample test ignores the hidden structure inside eval data. Since the question list is shared across models, conducting a paired-differences test lets us eliminate the variance in question difficulty and focus on the variance in responses. In our paper, we show how the result

of a paired-differences test will be related to the Pearson correlation coefficient between two models' question scores. When the correlation coefficient is higher, the standard error of the mean difference will be smaller.

In practice, we find the correlation of question scores on popular evals between frontier models to be substantial—between 0.3 and 0.7 on a scale of −1 to +1. Put another way, frontier models have an overall tendency to get the same questions right and wrong. Paired-difference analysis thus represents a "free" variance reduction technique that is very well suited for AI model evals. Therefore, in the interest of extracting the clearest signal from the data, our paper recommends reporting pairwise information—mean differences, standard errors, confidence intervals, and correlations—whenever two or more models are being compared.

Recommendation #5: Use power analysis

The flip side of the statistical significance coin is statistical power, which is the ability of a statistical test to detect a difference between two models, assuming such a difference exists. If an eval doesn't have very many questions, confidence intervals associated with any statistical tests will tend to be wide. This means that models will need to have a large underlying difference in capabilities in order to register a statistically significant result—and that small differences will likely go undetected. Power analysis refers to the mathematical relationship between observation count, statistical power, the false positive rate, and the effect size of interest.

In our paper, we show how to apply concepts from power analysis to evals. Specifically, we show researchers how to formulate a hypothesis (such as *Model A outperforms Model B by 3 percentage points*) and calculate the number of questions that an eval should have in order to test this hypothesis against the null hypothesis (such as *Model A and Model B are tied*).

We believe that power analysis will prove helpful to researchers in a number of situations. Our power formula will inform evaluators of models about the number of times to re-sample answers from questions (see Recommendation #3 above), as well as the number of questions that may be included in a random subsample while retaining the desired power properties. Researchers might use the power formula to conclude that an eval with a limited number of available questions is not worth running on a particular pair of models. Developers of new evals may wish to use the formula to help decide how many questions to include.

Conclusion

Statistics is the science of measurement in the presence of noise. Evals present a number of practical challenges, and a true science of evals remains underdeveloped. Statistics can only form one aspect of a science of evals—but a critical one, as an empirical science is only as good

Related posts:

The future of enterprise software

AARON LEVIE · 28 JAN 2026 · SOURCE

One of the biggest topics right now in the tech world is what does the future of software look like when AI agents are doing a substantial amount of work in the enterprise. Does software get compressed and simply become a database that agents leverage? Do we use AI agents to code all our own personalized enterprise software in the future? Do startups or incumbents win this new battle? How big are software markets in a world of agents using these tools? And much much more.

It's impossible to try and tackle all of these questions sufficiently in one sitting, but I figured I'd lay out a bit of a roadmap for what enterprise software looks like in the future.

Why companies buy software

Firstly, we should all get on the same page about why companies use and buy enterprise software in the first place. You can think of enterprise software as a codification of a company's processes. These are the parts of your company that you *don't* want to change spontaneously, because they provide the structure for your workflows and operations to keep your business running, or allow you to do the other work that really matters.

A large car company decides to procure an ERP system because it's the backbone for complex global operations where failures could mean far more than the cost of the system. It's responsible for keeping track of tens or hundreds of billions in revenue, and the company wants to ensure that every single transaction that they make goes through the exact same way every single time, with five 9s of uptime and reliability, and with an output that they can get accepted and approved by the auditors. Jobs are on the line, all the way up to the CEO, based on how well this system performs.

And the reason that companies choose to use software vendors that specialize in certain domains instead of just building this technology themselves can best be understood by the concept of "core" vs. "context," popularized by Geoffrey Moore. The core stuff is what makes your

If you've noticed something is going wrong, whether technically or communication wise, get those conflicts out in the open. Letting them stew never helps.

45. **I consistently interact with other professionals, professionally.**

    Be courteous and professional! Mike jokes about setting an incredibly low bar: no yelling, no hitting, …

46. **I can articulate what I believe others should expect from me.**

    Have a standard for yourself and be ready to tell your co-workers what it is.

47. **I do not require multiple reminders to respond to a request or complete work.**

    "Waiting for someone else to poke you is not an effective way to get your job done."

48. **I pursue opportunities to return value to the commons (when appropriate).**

    All our work builds on top of the work of countless others. At some point, you'll have opportunities to give back to the community at large. For example, talking at meetups, making open source contributions, or even just discussing topics with your team to boost everyone's skills.

49. **I actively work to bring value to the people I work with.**

    You're part of a team, so work to help them.

50. **I actively work to ensure under-represented voices are heard.**

    Don't stand by leaving this to be someone else's problem. Do something to make sure that minorities are heard. This might mean ensuring that the minority person at work gets a chance to speak, that your hiring process is unbiased, or that your website is accessible for users who rely on screen readers.

Fin

Let's learn and grow together,

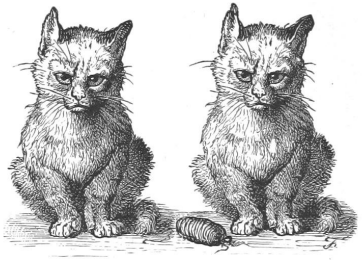
—Adam

One summary email a week, no spam, I pinky promise.

as its measuring tools. We hope that the recommendations in our paper **Adding Error Bars to Evals: A Statistical Approach to Language Model Evaluations** will help AI researchers calculate, interpret, and communicate eval numbers with greater precision and clarity than before—and we encourage researchers in the AI community to explore other techniques from experiment design so that they may understand more exactly all the things that they want to measure.

Mike Acton's Expectations of Professional Software Engineers

ADAM JOHNSON · 26 APR 2026 · SOURCE



grumpy cats ranting

In a 2019 talk/rant titled "Everyone Watching This Is Fired", games industry veteran Mike Acton rattled off a sample of 50 things he expects of developers he works with. The title refers to his tongue-in-cheek suggestion that anyone who doesn't meet all these requirements would be immediately fired.

Although his sense of humour isn't for everyone, the suggestions are valuable. The list forms a baseline for software engineers to compare themselves against, and it's not very specific to the games industry.

I couldn't find a textual version, so I've written up the list below, with my own expansions of each point and some further quotes from Mike. I hope this list inspires you to improve as an engineer, as it has me.

1. **I can articulate precisely what problem I am trying to solve.**

It's all too easy to get stuck in the weeds and lose track of why you're doing what you're doing. Keep top of mind what the actual end goal is do, and you might spot an alternative path.

2. **I have articulated precisely what problem I am trying to solve.**

   Communicate the problem "out loud" to other team members, your product manager, etc.

3. **I have confirmed that someone else can articulate what problem I am trying to solve.**

   Communication! Ensure your team is all on the same page. Make sure your understanding of the problem is complete.

4. **I can articulate why my problem is important to solve.**

   If you solve the problem you're working on, who benefits, and how much?

5. **I can articulate how much my problem is worth solving.**

   If you say it's worth "as long as it takes"… Mike does not have friendly words for you. For *any* problem there's a maximum amount of time and effort worth investing in solving it. At least have some idea of the upper bound.

6. **I have a Plan B in case my solution to my current problem doesn't work.**

   Imagine you're days or hours before the deadline, and you can tell that completing Plan A will be impossible. What do you do instead? Maybe you have a simplified algorithm, or you can disable a certain subsystem. Have more than one plan.

7. **I have already implemented my Plan B in case my solution to my current problem doesn't work.**

   Mitigate risk by writing the backup version first. This means you always have a safety net and you can learn more about the problem space in order to iterate on Plan A.

8. **I can articulate the steps required to solve my current problem.**

   Programming only works when you break down problems into manageable chunks. Have a sketch of the steps to the end state before you begin.

9. **I can clearly articulate unknowns and risks associated with my current problem.**

   Data output is also rarely optimal. Are you frequently writing out data that hasn't changed? Are you writing many fields when only one is used downstream? Again, know about it so you can reason about it.

37. **I can articulate how all the data I use is laid out in memory.**

    Many programming languages and frameworks can handle memory for you, but that doesn't abdicate you of responsibility. Know how your tools lay out memory, so you can tell when another approach makes sense.

    For example, in Python most objects are based on dictionaries, so you should have a solid understanding of how they work, and alternatives like slotted classes or arrays.

38. **I never use the phrase "platform independent" when referring to my work.**

    Any system depends on many things below it. Know what they are.

39. **I never use the phrase "future proof" when referring to my work.**

    Future-proofing is "100% a fool's errand". "You can't pre-solve problems you have no information of."

40. **I can schedule my own time well.**

    "You're an adult person, just use a calendar."

41. **I am vigilant about not wasting others' time.**

    Don't waste time asking questions that you can google in five seconds. But also don't waste loads of time struggling for days alone when you could get help from a team member in minutes! Find the balance.

42. **I actively seek constructive feedback and take it seriously.**

    Ask for feedback and do something about it.

43. **I am not actively avoiding any uncomfortable (professional) conversations.**

    If there's something wrong at work, don't put off talking about it.

44. **I am not actively avoiding any (professional) conflicts.**

There's no perfect profiling tool, so know how to use more than one.

For example, some great Python profilers are cProfile, py-spy, Austin, Scalene, Fil, and memray. They all have different characteristics and complement each other.

31. **I know how to significantly improve the performance of my system without changing the input/output interface of the system.**

    Do you know the next step to optimize your system? You don't have to do it right now, as it may not be worth it, but you should have an idea what you'd do next to make your code faster. For example, use a faster but less convenient data structure, or convert a hot function into a faster language (such as Python to C with Cython).

    This should also guide you to designing interfaces that are optimizable in the first place. For example, don't commit to returning expensive-to-compute results immediately, but instead return a promise.

32. **I know specifically how I can and will debug live release builds of my work when they fail.**

    Bugs are inevitable. You should know the tools that will let you work through those problems in production, such as logs, debuggers, or a live shell.

33. **I know what data I am reading as part of my solution and where it comes from.**

    Know where the data comes from, in what format, and how you can read it.

34. **I know how often I am reading data I do not need as part of my solution.**

    Data access is rarely optimal. You'll often be moving data that's not required for your solution, such as unnecessary fields or wrapper objects. If you don't know about this waste, you can't reason about whether it's worth the overhead or not.

35. **I know what data I am writing as part of my solution and where it is used.**

    All output data is intended for use by a human or another program. Be organized enough to know what the downstream consumers of your output are.

36. **I know how often I am writing data I do not need to as part of my solution.**

    There are always going to be things you don't know. You should know where they are in your plan, so you can manage them.

10. **I have not thought or said "I can just make up the time" without immediately talking to someone.**

    Say it's Wednesday, you have a project due on Friday, and you get some new task dropped on your lap. You think "I'll do the new thing now, and make up the time for the original task by Friday"… mistake! Communicate about the conflict on Wednesday. Your product manager will help manage the timing and risk.

11. **I write a "framework" and have used it multiple times to actually solve a problem it was intended to solve.**

    If you're writing a tool of some kind, you should verify it works in practice. Too often people create something in isolation and it doesn't end up delivering in the real world.

    (This is how Django came to be: from a real team making real websites, on deadlines!)

12. **I can articulate what the test for completion of my current problem is.**

    If you don't know when to stop, you might find yourself going down rabbit holes, chasing unimportant marginal gains.

13. **I can articulate the hypothesis related to my problem and how I could falsify it.**

    If a hypothesis cannot be proven wrong, there's no knowledge to be gained. As Karl Popper showed, science only works through falsification.

14. **I can articulate the (various) latency requirements for my current problem.**

    Any time you write code, you should consider when the output data is required. Not every caller needs output data instantly, and nor do you have an unbounded amount of time to perform everything. At least get an idea of the sensible bounds.

15. **I can articulate the (various) throughput requirements for my current problem.**

    How much data needs to come through the system? How many bytes, requests, or frames per second?

16. **I can articulate the most common concrete use case of the system I am developing.**

You should know what actual users of your system will actually be doing most of the time. Having a vague idea doesn't help, since knowing which pattersn are common informs which way to write a given piece of code.

17. **I know the most common actual, real-life values of the data I am transforming.**

    Beyond use cases, you should know the data inside the system. For example, if your function works with integers, you'd probably write it quite differently if 99% of the values are 0.

18. **I know the acceptable ranges of values of all the data I am transforming.**

    Computer systems always have limits. Know the ranges for the data types you're working with (and enforce them).

19. **I can articulate what will happen when (somehow) data outside that range enters the system.**

    Murphy's Law says "anything that can go wrong will go wrong". Know how your system will behave in such cases, and handle such problems if necessary.

20. **I can articulate a list of input data into my system roughly sorted by likelihood.**

    Have an idea of the space of possible data, what's most likely, second most likely, etc. Code appropriately, for example checking for common error conditions first.

21. **I know the frequency of change of the actual, real-life values of the data I am transforming.**

    Reason about the frequency of change and figure out how often you'll want to calculate derived values.

22. **I have (at least partially) read the (available) documentation for the hardware, platform, and tools I use most commonly.**

    Read the friendly manual! Go a step beyond day-to-day reference, and try reading the full documentation to gain a deep understanding.

    (Jens Oliver Meiert calls reading the HTML specification the Web Developer's Pilgrimage.)

23. **I have sat and watched an actual user of my system.**

    Watching users can massively break shift your view of how your software works. Do it!

24. **I know the slowest part of the users of my system's workflow with high confidence.**

    Any workflow has a bottleneck. Make sure you know what it is so you can focus efforts there, if need be.

25. **I know what information users of my system will need to make effective use of the solution.**

    Think about what documentation or data users need to understand and use your solution.

26. **I can articulate the finite set of hardware I am designing my solution to work for.**

    Software requires hardware. Know what hardware your program targets, such as:

    - CPU architectures
    - Minimum requirements for memory, CPU speed, and network bandwidth
    - Input devices
    - Output devices
    - The environment the hardware runs in (e.g. data centre or living room)

27. **I can articulate how that set of hardware specifically affects the design of my system.**

    If you're targetting low end devices, how do you ensure you don't exhaust memory? If some users don't have pointing devices, how do you accommodate them?

28. **I have recently profiled the performance of my system.**

    If you're developing a local app, run profiling tools regularly to gain an idea of performance over time. With server based programs, you can install an APM (Application Performance Monitoring) tool in production and have continuous profiling data.

29. **I have recently profiled memory usage of my system.**

    Make sure you aren't wasting memory.

30. **I have used multiple different profiling methods to measure the performance of my system.**